

IA Aplicada com LLMs

Aula 02 — Custo, latência e a decisão

Onde paramos

Os três decks anteriores deram os eixos **qualitativos**: o que diferencia um modelo, onde ele roda, o que cada parâmetro faz, como controlar a saída.

Falta a parte que **decide** — e que quase nenhum curso ensina:

a conta.

Por que isso é diferente de outro software

Um sistema de LLM tem uma característica incomum:

o custo marginal por requisição não é desprezível.

Uma API REST comum custa frações de centavo por chamada, e ninguém pensa nisso.

Uma chamada de LLM pode custar **mil vezes mais** — e um agente faz várias por tarefa.

Custo deixa de ser assunto de infraestrutura e vira requisito de projeto.

Aviso sobre preços

Nenhum número de preço neste deck é confiável.

Não porque esteja errado hoje — porque **preço de LLM muda**.

Todos os valores são **ilustrativos**, para acompanhar a aritmética.

Os preços reais estão em **mistral.ai/pricing** — e é de lá que você tira, **anotando a data**.

Roteiro

1. **A unidade de cobrança** — e por que entrada e saída têm preços diferentes
2. **A conta** — do token ao mês
3. **Onde o custo mora** — por tipo de arquitetura
4. **As alavancas** — em ordem de impacto
5. **Latência** — três números diferentes
6. **Benchmark próprio** — e por que leaderboard não decide
7. **A matriz de decisão**

Objetivos

- **Calcular** custo por requisição e projetar o mensal
- **Explicar** qual tarifa domina em RAG, geração longa e agentes
- **Distinguir** TTFT, tokens/s e tempo total
- **Argumentar** por que leaderboard não substitui benchmark próprio
- **Montar** uma matriz de decisão de modelo

Parte 1

A unidade de cobrança

Duas tarifas

	O que é	Preço	Por que
Entrada	system prompt, histórico, documentos, schema	mais barato	processados em paralelo , numa passagem (<i>prefill</i>)
Saída	tudo que o modelo gera	3 a 5× mais caro	gerados um de cada vez

A assimetria não é comercial, é **física**: o *prefill* aproveita o paralelismo da GPU; a geração é serial por natureza.

O que isso orienta no projeto

Encher o prompt é relativamente barato.

Deixar o modelo escrever muito é caro.

- resumo de 50 palavras a partir de um documento longo → custa pouco
- redação de 2.000 palavras a partir de um prompt curto → custa muito

A verdade vem na resposta

```
uso = resposta.usage  
print(uso.prompt_tokens, uso.completion_tokens, uso.total_tokens)
```

tiktoken serve para **planejar**.

usage é o que foi **cobrado**.

Não confunda os dois.

Parte 2

A conta

A fórmula

$$\text{custo} = \frac{T_{\text{in}} \times P_{\text{in}} + T_{\text{out}} \times P_{\text{out}}}{1.000.000}$$

Preços são publicados por **1 milhão de tokens**.

Passo a passo

Assistente de suporte · $P_{in} = 0,20$ · $P_{out} = 0,60$ (ilustrativos)

1. Somar por tarifa:

system 400 + histórico 1.200 + pergunta 100 = **1.700 de entrada** · **300 de saída**

2. Custo da requisição:

$$\frac{1.700 \times 0,20}{10^6} + \frac{300 \times 0,60}{10^6} = 0,00034 + 0,00018 = \mathbf{0,00052}$$

Meio milésimo de dólar. **É aqui que mora o erro de julgamento.**

Passo 3: multiplicar pelo volume

10.000 requisições/dia:

$$0,00052 \times 10.000 = 5,20/\text{dia} \approx \mathbf{156 \text{ US\$/mês}}$$

500 mil/dia: **~7.800 US\$/mês**

E se alguém aumentar o system prompt de 400 para 2.000 tokens "para melhorar as respostas", a entrada quase dobra — e a conta vai junto.

Sempre multiplique pelo volume. Sempre projete o crescimento.

Parte 3

Onde o custo mora

Por tipo de arquitetura

Aplicação	O que domina	Onde economizar
Classificação / extração	nada; é barato	modelo menor
RAG	entrada (documentos)	recuperar menos e melhor
Geração longa	saída	pedir saída mais curta
Chat longo	entrada, e cresce sozinha	resumir o histórico
Agente	tudo × N passos	menos passos, modelo menor nos fáceis

Chat longo cresce quadraticamente

O modelo não tem estado → você reenvia a conversa **inteira** a cada turno.

Turno 1 manda 1 mensagem. Turno 10 manda 19.

$$\text{custo total de } n \text{ turnos} \propto n^2$$

Uma conversa de 50 turnos custa **muito** mais que duas de 25.

Agente: o mesmo crescimento, com prompts maiores — por isso o **teto de passos** é, antes de tudo, um controle de custo.

Parte 4

As alavancas

Em ordem de impacto

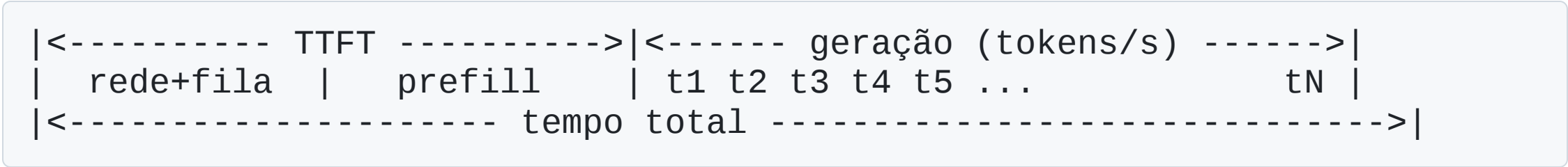
1. **Trocar de modelo** — a mais forte, de longe: **1 a 2 ordens de grandeza**
2. **Encurtar o prompt** — cada token cortado é cortado **em toda requisição, para sempre**
3. **Limitar a saída** — `max_tokens` e instrução de tamanho; é a tarifa cara
4. **Prompt caching** — desconto grande na parte fixa; exige o **estável no início**
5. **Batch** — desconto para o que não é interativo
6. **Cache de resultado** — pergunta repetida não precisa de LLM

1, 2 e 3 estão inteiramente sob seu controle. A maior parte do desperdício em sistemas de LLM é desperdício **simples**.

Parte 5

Latência

Três números, não um



Métrica	Afetada por	Importa quando
TTFT	tamanho do prompt , fila, tamanho do modelo	interface com usuário
Tokens/s	modelo, hardware, concorrência	respostas longas
Total	os dois + tamanho da resposta	lote, pipelines, agentes

Consequências práticas

- **Produto interativo** → otimize **TTFT** e use streaming. O total pode até piorar; a experiência melhora muito
- **Lote** → TTFT é irrelevante; o que importa é **throughput**, e paralelismo vale mais que velocidade individual
- **Agente** → o total é a **soma da trajetória**. 8 chamadas de 3 s = 24 s
- **Prompt grande piora o TTFT** — o *prefill* processa tudo antes do 1º token.
RAG com 20 documentos é lento **por construção**

Parte 6

Benchmark próprio

Medindo TTFT no código

```
inicio = time.perf_counter()
ttft = None
fluxo = client.chat.completions.create(..., stream=True,
                                         stream_options={"include_usage": True})

for evento in fluxo:
    if getattr(evento, "usage", None):
        uso = evento.usage # vem no último evento
    if evento.choices and evento.choices[0].delta.content:
        if ttft is None:
            ttft = time.perf_counter() - inicio # <- o primeiro token
```

Três cuidados ao ler o resultado

- 1. Uma medição não é uma medição.** Latência de API varia com fila, rede e hora do dia — ruído fácil de 2×. Repita 5 vezes e use a **mediana** (não a média, que uma execução lenta distorce).
- 2. Velocidade e custo estão medidos; qualidade não.** O script salva o texto para **você julgar**. Tabela de latência sem avaliação de qualidade é uma tabela sobre **quem erra mais rápido**.
- 3. O custo depende de você ter preenchido os preços.**

Leaderboard não decide nada

1. A tarefa não é a sua. Benchmarks medem múltipla escolha, matemática de competição, desafios de programação. O seu problema é *"extrair o número da nota fiscal deste PDF meia-boca"*.

2. Contaminação. Benchmarks públicos vazam para os dados de treino — o modelo pode ir bem porque **viu o teste**.

3. A média esconde a variância. `pass@1` $\neq$ `pass^k` : um agente que resolve 6 de 10 vezes é inútil para automação — e brilha no leaderboard.

4. Não mede custo e latência no seu contexto.

O que substitui

Um benchmark próprio de **5 a 20 tarefas reais** do seu domínio, com o resultado esperado anotado.

- leva algumas horas para montar
- é o **ativo mais valioso** do projeto
- responde a pergunta que interessa
- e **continua respondendo** quando um modelo novo sair

Você acabou de montar a primeira versão dele no laboratório desta aula.

Parte 7

A matriz de decisão

Os sete eixos

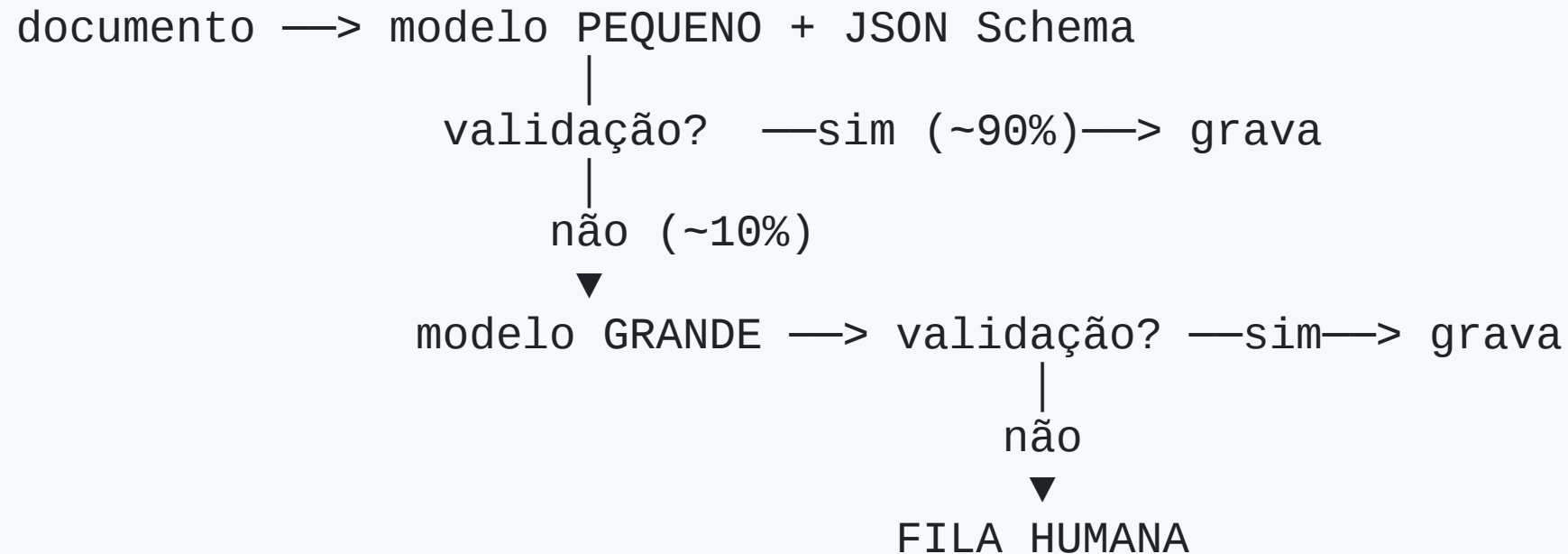
Eixo	Pergunta	Como obter
Qualidade	passa nas minhas N tarefas?	benchmark próprio + julgamento
Custo	US\$/requisição e /mês no meu volume	a conta da Parte 2
Latência	TTFT e total aceitáveis?	mediana de 5
Contexto	cabe meu maior prompt + resposta?	<code>models.list()</code>
Formato	suporta JSON Schema / tools?	catálogo + teste real
Dados	posso mandar esse conteúdo para lá?	política do provedor, jurídico
Estabilidade	versão fixável? depreciação?	documentação

Exemplo: nota fiscal, 50 mil docs/dia

Eixo	Pequeno	Grande	Decisão
Qualidade (20 notas reais)	18/20	20/20	grande ganha
Custo/mês	~US\$ 90	~US\$ 1.800	pequeno ganha por 20x
Latência (mediana)	0,8 s	2,4 s	pequeno ganha
Contexto · JSON · Dados	ok	ok	empate

Decisão: pequeno, com escalonamento. Resolve 90% por 5% do preço.

Os 2 casos que ele erra não somem



US\$ 90 + 10% do grande ≈ US\$ 270/mês em vez de US\$ 1.800.

O padrão mais rentável do curso

modelo pequeno + validação + escalonamento

Qualidade do grande **onde ela é necessária**, preço do pequeno no resto.

Reaparece em **agentes** (passos simples no modelo barato), em **RAG** (rerank barato antes do caro) e em **produção** (roteamento como política).

E note a terceira saída: **fila humana**. Todo sistema de LLM sério tem uma.

A pergunta não é "o modelo erra?" — ele erra. É "**o que acontece quando erra?**"

Arquitetura vale mais que parâmetro

Assistente sobre a documentação interna · 5.000 perguntas/dia:

Arquitetura	Entrada	Custo/mês
A — despejar o manual no prompt	80.050 tokens	~2.445
B — RAG: 5 trechos relevantes	3.050 tokens	~128
C — RAG + cache de repetidas (30%)	—	~90

De A para B: redução de **19×**. Não veio de trocar modelo nem de mexer em parâmetro — veio de **arquitetura**.

E o RAG provavelmente responde **melhor**: 80 mil tokens sofrem de *lost in the middle*.

Síntese (1/2)

- **Duas tarifas:** entrada barata (paralela, *prefill*), saída 3–5× mais cara (serial)
- O custo unitário **engana** — multiplique pelo volume e projete o crescimento
- `usage` é o cobrado; `tiktoken` é estimativa
- **RAG** gasta na entrada · **geração longa** na saída · **chat** cresce com n^2 · **agente** multiplica por N
- Alavancas: **trocar de modelo** (1–2 ordens) > encurtar prompt > limitar saída > caching > batch > cache de resultado

Síntese (2/2)

- Latência são **três números**: TTFT (o que o usuário sente), tokens/s, total.
Streaming melhora o **percebido**
- **Meça com repetição** — mediana de 5, não uma execução
- O script mede velocidade e custo; **qualidade quem julga é você**
- **Leaderboard não decide**: tarefa diferente, contaminação, `pass@1` $\neq$ `passk`
- **Benchmark próprio de 5–20 tarefas** é o ativo mais valioso — e a primeira suíte de **evals** do curso
- Defina o **limiar de qualidade antes** de ver os números
- **Pequeno + validação + escalonamento + fila humana** é o padrão mais rentável